

GoldVllm – Anleitung für Menschen

Stand: 25.09.2026 · gilt für den eingefrorenen GoldVllm-Stand (vLLM 0.29.0, Patch-Commit `d542745`, Modell-Revision `7b719225...`)

Diese Anleitung erklärt in normaler Sprache, **was GoldVllm ist, wie man es aufsetzt, wie man es betreibt und was man tut, wenn etwas klemmt**. Die technischen Einzelheiten stehen in den Originaldokumenten (`de/` und `docs/`). Hier steht, was man wirklich wissen muss.

0 Ganz kurz, ohne Fachbegriffe

Was ist das? Ein Rezept, mit dem eine künstliche Intelligenz (ähnlich wie ChatGPT) auf einem eigenen Rechner im Büro oder zu Hause läuft – ohne Cloud, ohne dass Daten das Haus verlassen, ohne monatliche Gebühren pro Anfrage.

Welcher Rechner? Ein bestimmter Kleinrechner von NVIDIA mit dem Chip „GB10“ (verkauft als DGX Spark oder ASUS Ascent GX10). Auf einem normalen PC oder Laptop funktioniert dieses Rezept **nicht**.

Was bringt das Rezept? Man könnte die KI auch mit Standard-Einstellungen starten. Wir haben aber über Tage gemessen, welche Einstellungen auf genau diesem Rechner am schnellsten **und** am stabilsten sind – und alles aufgeschrieben. Wer das Rezept nachkocht, bekommt dasselbe Ergebnis, ohne dieselben Fehler nochmals zu machen.

Was muss ich können? Für die Einrichtung (Kapitel 4) braucht man Grundkenntnisse im Linux-Terminal: Befehle kopieren, ausführen, Ausgaben lesen. Wer das nicht kann, gibt die Aufgabe am besten einem KI-Assistenten mit Terminalzugriff und legt ihm die Datei `Anleitung/AI/GoldVllm_Anleitung_KI.md` hin – dafür ist sie gemacht.

Welche Kapitel brauche ich?

Ich will ...	Lesen
nur verstehen, was das ist	Kapitel 0–2
es selbst einrichten	Kapitel 3–5
wissen, was bei einer Störung zu tun ist	Kapitel 6
selbst daran herumschrauben	Kapitel 7, danach <code>de/ERGEBNISSE.md</code> und <code>docs/LESSONS.md</code>

1 Worum geht es?

GoldVllm ist eine fertig eingestellte Konfiguration, mit der ein grosses Sprachmodell lokal auf **einem einzigen** NVIDIA-GB10-Rechner läuft (DGX Spark, ASUS Ascent GX10 oder baugleich, 128 GB Speicher).

Was	Wert
Modell	<code>RadixArk/Qwen3.8-Flash-Next-NVFP4</code> (135 GB auf der Platte)
Software, die das Modell ausführt	vLLM 0.29.0 plus Community-Patches von <code>blazux</code>
Kontext	bis 262'144 Tokens, also sehr lange Dokumente am Stück

Was	Wert
Zweck	lokales Gehirn für einen Agenten (bei uns „Ben“), der Werkzeuge aufruft, rund um die Uhr
Adresse im Betrieb	<code>http://127.0.0.1:8000</code> auf dem Gerät, OpenAI-kompatible Schnittstelle, mit API-Key

In einem Satz: 30 Agentenschritte in 62 Sekunden, rund 27 Wörterteile (Tokens) pro Sekunde, alle 12 Prüfaufgaben richtig, kein Absturz, kein Speicherfehler.

Warum „Gold“?

Weil dieser Stand gemessen, geprüft und **eingefroren** ist. Man verändert ihn nicht nebenbei. Wer etwas ausprobieren will, testet es getrennt und vergleicht es sauber mit diesem Stand (siehe Kapitel 7).

2 Die wichtigsten Begriffe

Begriff	Einfach erklärt
GB10 / Unified Memory	Grafikchip und Prozessor teilen sich denselben Speicher (128 GB). Läuft er voll, trifft es alles gleichzeitig – deshalb der RAM-Wächter
vLLM	das Programm, das das Modell lädt und Anfragen beantwortet
Container / Docker	vLLM läuft in einer abgeschotteten Box (Container). Das Rezept dafür heisst Image
KV-Cache	Zwischenspeicher für den gelesenen Text. Bei uns fest auf 12 GiB, damit genug Reserve bleibt
Prefix-Cache	Wenn der Agent denselben langen Anfang nochmals schickt, muss das Modell ihn nicht neu lesen. Macht Folgefragen bei 80k Text von 36 s auf 1.7 s schnell
MTP / spekulatives Decoding	Das Modell rät 2 Tokens im Voraus und prüft sie dann. Spart Zeit, wenn es oft richtig rät
Draft-Vokabular	Liste der Wörter, aus denen dieses Vorausraten wählt. Wir nutzen eine deutsche Liste – mit der englischen Standardliste rät das Modell auf Deutsch schlechter
PLE-Tabelle	48 GB grosser Teil des Modells, der direkt von der SSD gelesen wird statt im Speicher zu liegen. Nur deshalb passt das Modell überhaupt auf ein Gerät
Gates	Korrektheitsprüfung (<code>bench/gates.py</code>). „Server antwortet“ heisst noch nicht „Server rechnet richtig“
ABAB	Vergleichsmethode: alt, neu, alt, neu. Nur so sieht man, ob eine Änderung wirklich etwas bringt

3 Was man braucht

	Anforderung
Gerät	1× NVIDIA GB10 mit 128 GB, NVMe-SSD mit mindestens 200 GB frei
Betriebssystem	Ubuntu 24.04 (DGX-OS-Basis), getestet mit Kernel <code>7.0.0-1019-nvidia</code>
Treiber	NVIDIA 580.178.04, CUDA 13.0
Docker	29.1.3 mit <code>nvidia-container-toolkit</code> (Docker muss die GPU sehen)
Swap	16 GB empfohlen

	Anforderung
Zeit	rund 1 Stunde, das meiste davon ist der Modell-Download
Internet	nur für Aufbau und Download. Im Betrieb startet das Modell offline

Achtung beim GX10: Die NVIDIA-Treibermodule werden bei einem Kernel-Update nicht automatisch neu gebaut (kein DKMS). Nach einem Kernel-Update fehlen dann GPU und Bildschirm. **Kernel-Updates zurückhalten**, solange das nicht geklärt ist.

4 Einrichten – Schritt für Schritt

Alle Befehle laufen im Terminal des GB10. Der Repo-Ordner heisst hier `GoldVllm/`.

Schritt 1 – Prüfen, ob Docker die Grafikkarte sieht

```
docker run --rm --gpus all --entrypoint nvidia-smi vllm/vllm-openai:v0.29.0
```

Es muss eine Tabelle mit „GB10“ erscheinen. Wenn nicht: zuerst Treiber und nvidia-container-toolkit in Ordnung bringen.

Schritt 2 – Das Image bauen

```
git clone https://github.com/blazux/qwen3.8-Flash-DGX.git
cd qwen3.8-Flash-DGX
git checkout d542745
DOCKER_BUILDKIT=1 docker build -f Dockerfile.v0.29 -t gx10-vllm:goldvllm .
cd ..
```

Wichtig ist `DOCKER_BUILDKIT=1`. Der alte Builder las auf dem GX10 pro Schritt rund 30 GB und hätte etwa zwei Stunden gebraucht. (Wie lange der BuildKit-Bau dauert, haben wir nicht gemessen: Unser eigenes Image entstand aus demselben Rezept, abgetippt für einen einzigen Container, siehe `build/baue_b_inner.sh`, in 100 Sekunden.) Ein Neubau wird nie byte-gleich mit unserem Image, weil Zeitstempel einfließen – das ist normal. Kontrolle: `docker run --rm --entrypoint python3 gx10-vllm:goldvllm -c "import vllm; print(vllm.__version__)"` muss `0.29.0` ausgeben.

Schritt 3 – Das Modell herunterladen (135 GB)

```
pip install -U "huggingface_hub[cli]"
hf download RadixArk/Qwen3.8-Flash-Next-NVFP4 --revision 7b719225242aacd3dbd3f9407468c2ee9a9d2594
```

Genau diese Revision nehmen, keine neuere. Die Liste aller 419 Dateien mit Grösse steht in `data/model-snapshot-7b719225.tsv`.

Schritt 4 – Deutsches Vokabular und Schlüssel ablegen

```
sudo install -D -m 0444 draftvocab/draft_vocab_de_65536.npy /opt/gx10/draftvocab/draft_vocab_de_65536.npy
sha256sum /opt/gx10/draftvocab/draft_vocab_de_65536.npy
python3 -c 'import secrets;print(secrets.token_hex(32))' > ~/.config/vllm.key
chmod 600 ~/.config/vllm.key
```

Die Prüfsumme muss mit `a8647394...54a23a1` übereinstimmen. Der Schlüssel in `~/.config/vllm.key` ist das Passwort für die Schnittstelle – **nicht ins Git, nicht weitergeben**.

Schritt 5 – Starten

```
VOCAB=/opt/gx10/draftvocab/draft_vocab_de_65536.npy KEYFILE=~/.config/vllm.key ./config/run-goldvllm.sh
docker logs -f qwen38-flash
```

Jetzt **14 bis 15 Minuten warten**, bis im Log `Application startup complete` steht. Nicht abbrechen, auch wenn es hängt aussieht.

Das Skript bricht von sich aus ab, wenn etwas fehlt:

Meldung	Bedeutung
<code>KEYFILE fehlt</code>	Schritt 4 (Schlüssel) nachholen
<code>Modell-Revision ... fehlt</code>	Schritt 3 nicht fertig oder falsche Revision
<code>Draft-Vokabular fehlt oder SHA falsch</code>	Schritt 4 (Vokabular) prüfen
<code>GPU ist belegt</code>	Es läuft schon ein anderes Modell. Erst beenden – nie zwei gleichzeitig

Schritt 6 – Prüfen, ob es richtig rechnet

```
curl -s -o /dev/null -w '%{http_code}\n' localhost:8000/health # erwartet: 200
curl -s -o /dev/null -w '%{http_code}\n' localhost:8000/v1/models # erwartet: 401 (ohne Schlüssel
gespernt)
cd bench
VLLM_API_KEY=$(cat ~/.config/vllm.key) KORPUS_DIR=/pfad/zu/txt-dateien \
python3 gates.py --url http://127.0.0.1:8000 --label erster-start --ausgabe gates.json
```

Am Ende muss `GATES OK` stehen. `KORPUS_DIR` zeigt auf einen Ordner mit beliebigen längeren `.txt`-Dateien (z. B. Handbücher), daraus baut das Skript die langen Testdokumente.

Schritt 7 – Dauerbetrieb einrichten

Erst **RAM-Wächter**, dann den Autostart, dann die Recovery. Was die drei tun, steht in Kapitel 5. Die Befehle laufen im Repo-Ordner. Überall, wo `DEIN_NUTZER` steht, den eigenen Linux-Benutzernamen einsetzen.

```
# 1 feste Kopie für den Betrieb (nicht der Ordner, in dem man herumprobiert)
sudo cp -r . /opt/goldvllm

# 2 RAM-Wächter
sudo install -m 0755 config/schutz/llm-memory-guard.sh /usr/local/sbin/
sudo cp config/schutz/llm-memory-guard.service config/schutz/llm-memory-guard.timer /etc/systemd/system/

# 3 Autostart: vorher in der Datei DEIN_NUTZER ersetzen
sudo cp config/systemd/llm-server.service /etc/systemd/system/
sudo nano /etc/systemd/system/llm-server.service

# 4 Recovery: in /etc/default/llm-recovery DEIN_NUTZER ersetzen
sudo install -m 0755 config/schutz/llm-recovery.sh /usr/local/sbin/
sudo cp config/schutz/llm-recovery.service config/schutz/llm-recovery.timer /etc/systemd/system/
sudo install -m 0644 config/schutz/llm-recovery.default /etc/default/llm-recovery
sudo nano /etc/default/llm-recovery

# 5 einschalten
sudo systemctl daemon-reload
sudo systemctl enable --now llm-memory-guard.timer
sudo systemctl enable llm-server.service llm-recovery.timer
```

Danach **einmal neu starten** und prüfen, dass alles von selbst wieder hochkommt: 15 Minuten warten, dann nochmals Schritt 6. Läuft das Modell von Schritt 5 noch, zuerst `docker stop qwen38-flash` – sonst startet systemd ein zweites Mal auf eine belegte GPU.

5 Im Alltag

Wer steuert was?

- **systemd** startet und stoppt das Modell (`llm-server.service`). Docker startet selbst nichts neu.
- Der **RAM-Wächter** schaut alle 10 Sekunden auf den Speicher. Sind mehr als **122 GB belegt**, stoppt er alle Modellserver. So reisst ein voller Speicher nicht das ganze Gerät mit.
- Die **Recovery** holt das Modell nach einem Wächter-Stopp zurück – aber nur, wenn der Speicher wieder frei ist, die GPU gesund ist und höchstens 2 Versuche in 6 Stunden. Danach bleibt sie auf FAILED stehen, bis jemand nachschaut.

Nützliche Befehle

Wozu	Befehl
Läuft es?	<code>systemctl status llm-server</code> und <code>docker ps</code>
Log ansehen	<code>docker logs --tail 40 qwen38-flash</code>
Neu starten	<code>sudo systemctl restart llm-server</code> (danach 15 min warten)
Wächter-Meldungen	<code>journalctl -t llm-memory-guard --since today</code>
Recovery-Zustand	<code>sudo llm-recovery.sh --status</code>

Wozu	Befehl
Recovery nach FAILED freigeben	<code>sudo llm-recovery.sh --reset</code>
Recovery ausschalten	<code>sudo touch /etc/llm-recovery.disabled</code>
Speicher anschauen	<code>free -g</code>
Läuft noch ein anderes Modell?	<code>nvidia-smi --query-compute-apps=pid --format=csv,noheader</code> (muss leer sein, bevor man startet)

Was ist normal?

	normaler Wert
Startzeit nach Neustart	14–15 min, in dieser Zeit ist der Agent ohne Gehirn
Freier Speicher (MemAvailable) im Betrieb	etwa 15–16 GiB
Swap belegt	etwa 7 GB
GPU-Temperatur	bis 80 °C, keine Drosselung
Meldungen <code>NV_ERR_NO_MEMORY</code> im Kernel-Log beim Laden	harmlos, kein echter Fehler

6 Wenn etwas nicht stimmt

Beobachtung	Was tun
Modell startet nicht	<code>docker logs --tail 40 qwen38-flash</code> lesen. Nicht blind neu starten. Prüfen, ob die GPU frei ist und ob vor dem Start ≥ 90 GB Speicher frei sind
Wächter hat gestoppt	Zuerst den Grund finden: lief ein zweites Modell, ein Test, ein grosser Zusatzprozess? Erst dann neu starten
Antworten sind seltsam (Wiederholungen wie <code>!!!!</code> , leerer Text, falsche Zahlen)	Nicht weiter messen oder benutzen. <code>gates.py</code> laufen lassen. Bei Fehler: zurück auf den eingefrorenen Stand
Nach Neustart keine GPU, kein Bild	Wahrscheinlich Kernel-Update ohne passende NVIDIA-Module. Kernel zurück oder Module neu bauen
Recovery tut nichts	<code>sudo llm-recovery.sh --status</code> und <code>journalctl -t llm-recovery</code> ansehen. Steht dort „LLMR_KEYFILE nicht gesetzt“: <code>/etc/default/llm-recovery</code> ausfüllen. Bei FAILED Ursache klären, dann <code>--reset</code>
Recovery hält ein gesundes Modell für krank	den Befehl aus <code>LLMR_HEALTH_CMD</code> von Hand als <code>LLMR_HEALTH_USER</code> ausführen und schauen, woran er scheitert

Grundregel: `/health` sagt nur, dass der Server antwortet – nicht, dass er richtig rechnet. Im Zweifel immer `gates.py`.

7 Etwas ausprobieren, ohne den Gold-Stand zu gefährden

1. **Vorher sichern:** Image-ID, Startbefehl (`docker inspect`), jede Datei, die man ändert. Aufschreiben, wie man zurückkommt.
2. **Nur eine Sache ändern.** Nie zwei Einstellungen gleichzeitig.
3. **Nie zwei Modelle gleichzeitig laufen lassen.** Das Produktionsmodell für den Test stoppen.
4. **Testcontainer heissen** `bench-...` und laufen auf einem anderen Port.
5. **Vorher festlegen, ab wann es als Verbesserung gilt.** Dann ABAB messen (alt, neu, alt, neu).
6. **Unter 5 % ist Rauschen.** Einzelmessungen schwanken hier um ± 5 % (Decode) bis 13 % (Agentenläufe).
7. Am Ende läuft wieder der Gold-Stand und `gates.py` sagt `GATES OK` – ausser man hat bewusst anders entschieden.

Was wir schon getestet haben, muss man nicht nochmal machen:

Idee	Ergebnis
Offizielles vLLM 0.29 statt Preview-Build	grösster Gewinn: lange Texte 32–40 % schneller
KV-Cache fest auf 12 GiB statt „so viel wie passt“	stabil. Mit 18.7 GiB ist die Produktion einmal umgefallen
MTP 2 statt 1, 3 oder 4	2 ist am besten
Deutsches statt englisches Draft-Vokabular	+5 % Gesamtzeit, +8 % Schreibgeschwindigkeit
<code>FAST_ROWS=0</code> , MTP 3/4, deterministischer Kernel	schlechter oder ohne Nutzen – abgelehnt
Alte Tool-Ergebnisse aus dem Agentenkontext kürzen	kaum schneller, weil der Prefix-Cache das schon billig macht
SGLang statt vLLM	kurz und warm schneller, lange Texte langsamer, gesamt -0.7 %, harte Grenze bei 109–127k Tokens mit stiller leerer Antwort → verworfen

8 Grenzen – ehrlich

- Gemessen auf **einem** Gerät mit **einer** Arbeitslast (deutschsprachiger Agent, wenig gleichzeitige Nutzer).
- Das Image ist aus Community-Patches gebaut, **kein** offizielles NVIDIA- oder vLLM-Produkt. Ein Neubau ist nicht byte-gleich.
- Das Patch-Repo ist inzwischen weiter (vLLM 0.30). GoldVllm bleibt absichtlich beim getesteten Stand.
- Das deutsche Vokabular hilft **nur bei deutschem Text**. Für Englisch nicht gemessen.
- Die Recovery prüft nach einem Neustart nur, ob der Server antwortet und den Schlüssel akzeptiert. Ob er **richtig rechnet**, prüft sie nur, wenn man in `/etc/default/llm-recovery` zusätzlich `LLMR_HEALTH_CMD` einträgt (Beispiel mit `gates.py` steht dort).

9 Wo steht was im Repo?

Ordner/Datei	Inhalt
<code>README.md</code>	Übersicht (englisch)

Ordner/Datei	Inhalt
de/	alle Originaldokumente auf Deutsch: INSTALL, KONFIGURATION, BETRIEB, ERGEBNISSE, SGLANG
docs/	dieselben auf Englisch, plus LESSONS (was sich gelohnt hat)
config/run-goldvllm.sh	der Startbefehl
config/systemd/ , config/schutz/	Autostart, RAM-Wächter, Recovery
draftvocab/	deutsches Vokabular und wie man ein eigenes baut
bench/	Mess- und Prüfwerkzeuge (<code>gates.py</code> ist das wichtigste)
data/	alle Rohdaten der Messungen
Anleitung/AI/	die Fassung dieser Anleitung für KI-Assistenten